

Experiment no. 01

Title: DDA line drawing algorithm with slope(m) value and without slope(m) value.

Objective (with slope):

This algorithm is fundamental in computer graphics for rendering straight lines between two points in a raster display. By doing this algorithm implementation, we will be able to understand the principles of line drawing algorithms, slope considerations in line drawing (handling different cases of line slopes including positive, and negative slopes) & confidently implement the DDA algorithm & evaluate its performance relative to other line drawing methods.

Algorithm:

Firstly, we have to look at the slope m & the corresponding increments for x & y . The slope m is calculated as dy/dx , where $dx=x_1-x_0$ & $dy=y_1-y_0$.

Case 1: $0 \leq m \leq 1$ & $dx \geq 0$ & $dy \geq 0$

- This case applies when the line is going upwards to the right and the slope is between 0 and 1 (inclusive).
- **Sample figures:** $x_0=1, y_0=1, x_1=4, y_1=3$
- **Increments:** $x_increment=1, y_increment=m$

Case 2: $0 \leq m \leq 1$ & $dx < 0$ & $dy < 0$

- This case applies when the line is going downwards to the left and the slope is between 0 and 1 (inclusive).
- **Sample figures:** $x_0=4, y_0=3, x_1=1, y_1=1$
- **Increments:** $x_increment=-1, y_increment=-m$

Case 3: $-1 \leq m < 0$ & $dx > 0$ & $dy < 0$

- This case applies when the line is going downwards to the right and the slope is between -1 and 0 (inclusive).
- **Sample figures:** $x_0=1, y_0=3, x_1=4, y_1=1$
- **Increments:** $x_increment=1, y_increment=-m$

Case 4: $-1 \leq m < 0$ & $dx < 0$ & $dy > 0$

- This case applies when the line is going upwards to the left and the slope is between -1 and 0 (inclusive).
- **Sample figures:** $x_0=4, y_0=1, x_1=1, y_1=3$
- **Increments:** $x_increment=-1, y_increment=m$

Case 5: $m > 1$ & $dx > 0$ & $dy > 0$

- This case applies when the line is going steeply upwards to the right.
- **Sample figures:** $x_0=1, y_0=1, x_1=3, y_1=5$
- **Increments:** $x_increment=1/m, y_increment=1$

Case 6: $m>1$ & $dx<0$ & $dy<0$

- This case applies when the line is going steeply downwards to the left.
- **Sample figures:** $x_0=3, y_0=5, x_1=1, y_1=1$
- **Increments:** $x_increment=-(1/m), y_increment=-1$

Case 7: $m<-1$ & $dx>0$ & $dy<0$

- This case applies when the line is going steeply downwards to the right.
- **Sample figures:** $x_0=1, y_0=5, x_1=3, y_1=1$
- **Increments:** $x_increment=1/m, y_increment=-1$

Case 8: $m<-1$ & $dx<0$ & $dy>0$

- This case applies when the line is going steeply upwards to the left.
- **Sample figures:** $x_0=3, y_0=1, x_1=1, y_1=5$
- **Increments:** $x_increment=-(1/m), y_increment=1$

For each of these cases, the algorithm increments the x and y coordinates accordingly to draw the line pixel by pixel from (x_0, y_0) to (x_1, y_1) .

Code:

```
//Line Drawing using Slope
#include <bits/stdc++.h>
#include <GL/glut.h>
#include <windows.h>
using namespace std;

int x_0, y_0, x_1, y_1;
const int window_width = 300;
const int window_height = 300;

void draw_line() {
    float dx = x_1 - x_0;
    float dy = y_1 - y_0;
    int steps;
    if( abs(dx) > abs(dy) )
        steps=abs(dx);
    else
        steps=abs(dy);

    float m = dy / dx;
```

```

float m_abs = abs(m);
float x_increment, y_increment;

//case 1
if(m >= 0 && m <= 1 && dx >= 0 && dy >= 0){
    x_increment = 1;
    y_increment = m_abs;
}
//case 2
else if(m >= 0 && m <= 1 && dx < 0 && dy < 0){
    x_increment = -1;
    y_increment = -m_abs;
}
//case 3
else if(m >= -1 && m < 0 && dx > 0 && dy < 0){
    x_increment = 1;
    y_increment = -m_abs;
}
//case 4
else if(m >= -1 && m < 0 && dx < 0 && dy > 0){
    x_increment = -1;
    y_increment = m_abs;
}
//case 5
else if(m > 1 && dx > 0 && dy > 0){
    x_increment = 1 / m_abs;
    y_increment = 1;
}
//case 6
else if(m > 1 && dx < 0 && dy < 0){
    x_increment = -(1 / m_abs);
    y_increment = -1;
}
//case 7
else if(m < -1 && dx > 0 && dy < 0){
    x_increment = 1 / m_abs;
    y_increment = -1;
}
//case 8
else if(m < -1 && dx < 0 && dy > 0){
    x_increment = -(1 / m_abs);
    y_increment = 1;
}
glBegin(GL_POINTS);
float x = x_0;
float y = y_0;
for(int i = 0; i <= steps; i++){
    glVertex2i(x, y);
    cout << x << " " << y << endl;
    x += x_increment;
    y += y_increment;
}
glEnd();
}

```

```

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0);
    glPointSize(2.0);
    draw_line();
    glFlush();
}

void init(){
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, window_width, 0, window_height);
}

int main(int argc, char** argv){
    cout << "Enter the values of x_0 & y_0: ";
    cin >> x_0 >> y_0;
    cout << "Enter the values of x_1 & y_1: ";
    cin >> x_1 >> y_1;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(window_width, window_height);
    glutCreateWindow("DDA Line Drawing Algorithm");
    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}

```

Output:

```

Enter the values of x_0 & y_0: 4 15
Enter the values of x_1 & y_1: 20 1

```



Objective (without slope):

The objective of this algorithm to implement and understand the Digital Differential Analyzer (DDA) line drawing algorithm without explicitly using the slope (m). This algorithm is aimed at understanding the DDA algorithm (Grasping the fundamental concepts behind the DDA algorithm and how it approximates straight lines on raster displays), slope handling (learning to handle different cases of line slopes i.e. positive, negative without directly calculating the slope), computational efficiency (understanding the

efficiency of the DDA algorithm in terms of computational steps and resource utilization), practical application (gaining practical experience in rendering lines in computer graphics, which is foundational for more advanced graphics programming).

Algorithm:

The Digital Differential Analyzer (DDA) algorithm generates points between two given points to form a straight line. The implementation avoids directly using the slope and instead uses incremental steps based on the differences in x and y coordinates.

Algorithm Steps:

1. Calculate Differences: Determine the differences in the x and y coordinates:

$$dx = x_1 - x_0 \text{ \& } dy = y_1 - y_0$$

2. Determine Steps: Calculate the number of steps required to draw the line. This is the maximum of the absolute differences in the x or y directions:

$$\text{steps} = \max(|dx|, |dy|)$$

3. Calculate Increments: Determine the increment values for x and y for each step:

$$x_increment = dx / \text{steps}$$

$$y_increment = dy / \text{steps}$$

4. Initialize Starting Point: Set the starting point:

$$x = x_0 \text{ \& } y = y_0$$

5. Draw Points: Iterate through the number of steps, incrementing x and y by their respective increments, and plot the points:

for $i=0$ to steps:

- plot(x, y)
- $x += x_increment$
- $y += y_increment$

Code:

```

// without m
#include<iostream>
#include<GL/glut.h>
#include<windows.h>
using namespace std;
const int window_width = 300;
const int window_height = 300;
int x_0, y_0, x_1, y_1;
void draw_line()
{
    float dx = x_1 - x_0;
    float dy = y_1 - y_0;

    int steps;
    if( abs(dx) > abs(dy) )
        steps=abs(dx);
    else
        steps=abs(dy);

    float x_increment = dx / steps;
    float y_increment = dy / steps;

    float x = x_0;
    float y = y_0;

    glBegin(GL_POINTS);
    for(int i=0; i<=steps; i++){
        glVertex_2i(x, y);
        x += x_increment;
        y += y_increment;
    }
    glEnd();
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0);
    glPointSize(2.0);
    draw_line();
    glFlush();
}

void init()
{
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, window_width, 0, window_height);
}

```

```

int main(int argc, char** argv)
{
    cout << "Enter the value of x_0 and y_0 : " << endl;
    cin >> x_0 >> y_0;
    cout << "Enter the value of x_1 and y_1: " << endl;
    cin >> x_1 >> y_1;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(window_width, window_height);
    glutCreateWindow("DDA Line Drawing Algorithm without slope");
    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}

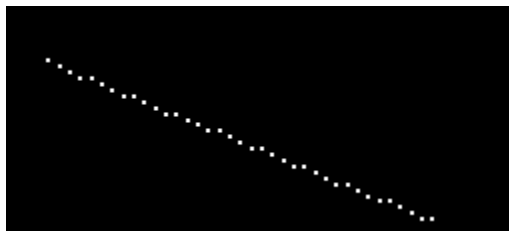
```

Output:

```

Enter the values of x_0 & y_0: 4 30
Enter the values of x_1 & y_1: 40 3

```



Discussion:

Some mistakes occurred during coding:

- Incorrect function names, so that I need to see this names repeatedly & find these mistakes.
- Sometimes, I can't properly handle floating point & round of numbers.
- The incrementing logic calculation issues.
- Sometimes, I have to use floating point variables but does some mistake to initialize variables.
- Some wrong loop boundary conditions.
- To handle special cases for slope vale, dx & dy.

Experiment no. 02

Title: Bresenham's line drawing algorithm

Objective:

The objective of this algorithm is to implement and understand Bresenham's line drawing algorithm, a key technique in computer graphics for efficiently rendering straight lines on raster displays. By exploring Bresenham's algorithm, we will learn how it optimizes line drawing by using integer arithmetic rather than floating-point calculations, thus enhancing computational efficiency and accuracy. This aims to provide a deep understanding of the algorithm's decision-making process for pixel selection, enabling us to grasp the concepts of incremental error and the handling of various line slopes and directions. Through coding practice, we will gain insights into the practical applications of Bresenham's algorithm in real-time graphics and visual computing. This also includes comparing Bresenham's approach with other line drawing algorithms, highlighting its advantages in terms of performance and precision.

Algorithm:

The algorithm steps of Bresenham's Line Drawing Algorithm are:

- **Input Coordinates:** Obtain the starting point (x_0, y_0) and the ending point (x_1, y_1) from the user.
- **Calculate Differences:** Compute the differences $dx = |x_1 - x_0|$ and $dy = |y_1 - y_0|$ between the x and y coordinates of the two points.
- **Initialize Variables:** Initialize variables x , y , x_{end} , y_{end} , $x_{increment}$, and $y_{increment}$ for iteration along the line.
- **Calculate Slope:** Determine the absolute slope $m = |dy / dx|$ of the line.
- **Select Increment Direction:** Based on the slope value, decide whether to increment x or y in each step of the algorithm.
- **Handle Slope < 1 :** If $m < 1$, set up the initial value of the decision parameter p and iterate along the x-axis. Update p and the current pixel position (x, y) based on the decision parameter.
- **Handle Slope ≥ 1 :** If $m \geq 1$, set up the initial value of p and iterate along the y-axis. Update p and the current pixel position (x, y) based on the decision parameter.
- **Plot Pixels:** Within the iteration loop, plot each calculated pixel position (x, y) using `glVertex2i(x, y)` to draw the line segment.
- **Display the Line:** In the display function, clear the screen, set the point size and color, call the Bresenham line drawing function, and flush the buffer to display the line.
- **Initialize Graphics Environment:** Initialize the OpenGL environment, including window size, position, display mode, and title.

- **Main Loop:** Start the main loop using `glutMainLoop()` to handle events and update the display continuously.

Code:

```
#include <bits/stdc++.h>
#ifdef __APPLE__
#else
#include <GL/glut.h>
#endif
#include <stdlib.h>
using namespace std;

int x_0, y_0, x_1, y_1;

void bresenhamLineDraw() {
    int dx = abs(x_1 - x_0);
    int dy = abs(y_1 - y_0);
    int dx_0 = x_1 - x_0;
    int dy_0 = y_1 - y_0;
    int x, y, xEnd, yEnd, x_increment, y_increment;

    float m = abs(dy / dx);

    if(m < 1){
        int p = 2 * dy - dx;
        int twoDy = 2 * dy;
        int twoDyDx = 2 * (dy - dx);
        y_increment = 1;
        if(dx_0 < 0 && dy_0 < 0){
            dy_0 = 1;
            dx_0 = 1;
        }
        if(dx_0 < 0 || dy_0 < 0){
            y_increment = -1;
        }

        if (x_0 > x_1){
            x = x_1;
            y = y_1;
            xEnd = x_0;
        }
        else{
            x = x_0;
            y = y_0;
            xEnd = x_1;
        }
        glBegin(GL_POINTS);
        glVertex2i(x, y);
        while (x < xEnd){
            x++;
            if (p < 0) {
                p += twoDy;
            }
        }
    }
```

```

        else {
            y += y_increment;
            p += twoDxDy;
        }
        glVertex2i(x, y);
        cout << x << " " << y << endl;
    }
    glEnd();
}
else{
    int p = 2 * dx - dy;
    int twoDx = 2 * dx;
    int twoDxDy = 2 * (dx - dy);
    x_increment = 1;
    if(dx_0 < 0 && dy_0 < 0){
        dy_0 = 1;
        dx_0 = 1;
    }
    if(dx_0 < 0 || dy_0 < 0){
        x_increment = -1;
    }
    if (y_0 > y_1){
        x = x_1;
        y = y_1;
        yEnd = y_0;
    }
    else{
        x = x_0;
        y = y_0;
        yEnd = y_1;
    }
    glBegin(GL_POINTS);
    glVertex2i(x, y);
    while (y < yEnd){
        y++;
        if (p < 0) {
            p += twoDx;
        } else {
            x += x_increment;
            p += twoDxDy;
        }
        glVertex2i(x, y);
        cout << x << " " << y << endl;
    }
    glEnd();
}
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(2.0);
    glColor3f(1.0, 1.0, 1.0);
    bresenhamLineDraw();
    glFlush();
}

```

```

void init(int windowWidth, int windowHeight){
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, windowWidth, 0, windowHeight);
}

int main(int argc, char *argv[]){
    cout << "Enter the values of x_0 and y_0: " << endl;
    cin >> x_0 >> y_0;
    cout << "Enter the values of x_1 and y_1: " << endl;
    cin >> x_1 >> y_1;

    glutInit(&argc, argv);
    glutInitWindowSize(400, 400);
    glutInitWindowPosition(10, 10);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutCreateWindow("Bresenham Line Drawing Algorithm");
    init(640, 480);
    glutDisplayFunc(display);
    glutMainLoop();

    return EXIT_SUCCESS;
}

```

Output:

```

Enter the values of x_0 and y_0:
4 100
Enter the values of x_1 and y_1:
90 6

```



Discussion:

There are several common mistakes that I have done during this implementation. Here are a few:

- Forgetting to initialize variables before using them.
- Incorrect logic implementation of algorithms with the round up & variable types.
- Sometimes failed to handle boundary conditions of loops.
- write code without comments, then forget what I have done by doing these.